CSCI 580 - Artificial Intelligence
Assignment #6
Perceptron $\longrightarrow$ Neuron

Joseph Lorenzo Bautista

April 17th, 2026

Part #1: Implementing Perceptron via Heuristic Approach

Plotting data from `data.csv`, we implement perceptron using the *Heuristic* approach. Here, we plot the initial separation line as `red`, subsequent ones after each iteration in `dashed green`, and the last one in `black`. After implementing, `perceptron_heu.py` was produced. Here we trained and ran with a set epoch value of 25 but differing learning rates of values 0.001, 0.1 and 1.

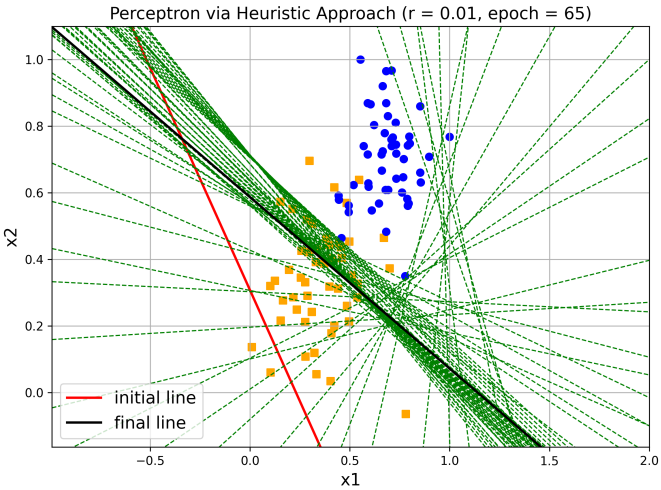
Including snippets of the code from `perceptron_heu.py`. The follow is the initialization of random weights and bias, classification of each point, and the actual training loop with heuristic updates.

```python
# Step #1: starting with random weights and bias
random.seed(0)
w1  = random.uniform(-1, 1)
w2  = random.uniform(-1, 1)
b   = random.uniform(-1, 1)

r = 1
num_epochs = 65

# Helper that classifies one point
def classify(x1, x2, w1, w2, b):
    if w1 * x1 + w2 * x2 + b >= 0:
        return 1
    else:
        return 0
```

```python
# Step #2 and Step #3: training the perceptron
for epoch in range(num_epochs):
    for i in range(n_points):
        x1 = x1_list[i]
        x2 = x2_list[i]
        y = labels[i]

        guess = classify(x1, x2, w1, w2, b)

        if guess != y:
            if guess == 0:

                b = b + r
                w1 = w1 + r * x1
                w2 = w2 + r * x2
            else:
                b = b - r
                w1 = w1 - r * x1
                w2 = w2 - r * x2
```

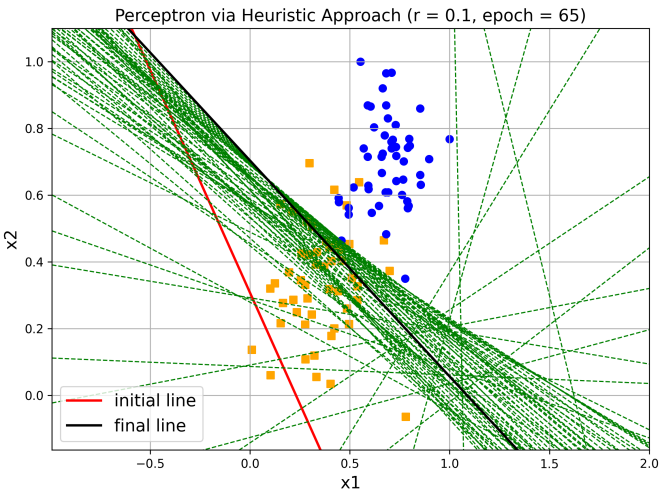
With these two snippets, the heuristic approach starts with random weights and bias, and then loops through all of the data points multiple times. Each point then gets a binary 0 or 1 classification, and if wrong, the weights are moved/changed a bit by a fixed amount towards the correct answer. And if its right, then nothing happens. And this process repeats for a set number of epochs until the separation line establishes itself into place.

Starting with $r = 0.01$ and $epoch = 65$, here we can see that the initial red line, which was placed randomly, did not clearly separate the two classes. From there, the algorithm iteratively adjusted and changed its weights when it misclassified a point, which produced the green dashed lines as the boundary rotated to find a better fit. Additionally, the black lines passes through the two cluster points, which correctly classifies the majority of the blue circles as being on one side, and the orange squares on the other. Here, only a handful of points were misclassified near the boundary.



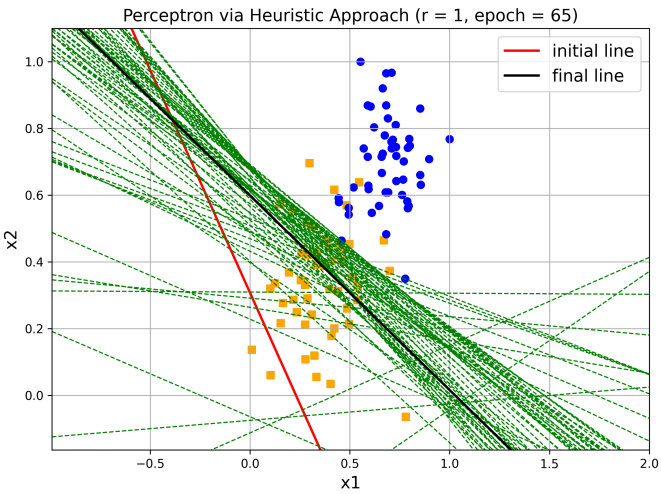
Because the two classes aren't perfectly linearly separated, the algorithm never fully converges and the green lines are seen to be fanning out instead of stacking together.

Next, with $r = 0.1$ and $epoch = 65$, here we can see that starting on the initial red line, the algorithm was more aggressive on each misclassified points, making a more compact group of dashed green lines. The final black lines again, separates most of the blue circles from orange squares.



Additionally, with a larger learning rate of $r = 0.1$, the boundary jumps further on each iteration/update, which leads to more converging together tightly and clustering around the black line with only a few outliers. But like the previous learning rate value, the data is not linearly separable, which means that the algorithm doesn't fully converge.

And now with $r = 1$ and $epoch = 65$, here we can see that with the same initial red line, the algorithm was even more aggressive on each misclassified points, but led to a more compact group of dashed green lines. The black line is the same as the previous learning rates, separating the two data points with a few outliers.



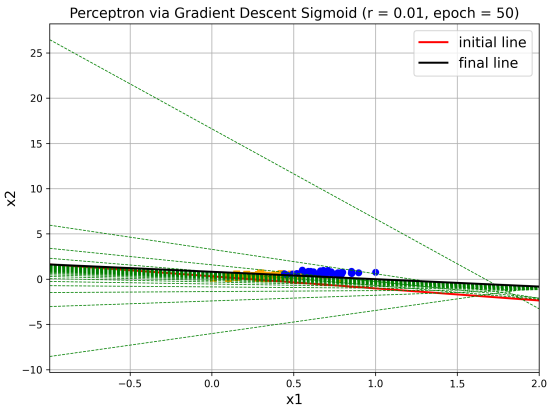
With such a large step size amount compared to the previous 2, the boundary swings a lot more on each iteration/update instead of staying still, which we can see by the green lines that are extending far from the final position. And since the data is not linearly separable, the oscillation is more pronounced here, and the lines are overshooting repeatedly. Here, we can see that a larger learning rate converges faster initially but is met with a trade off of instability.

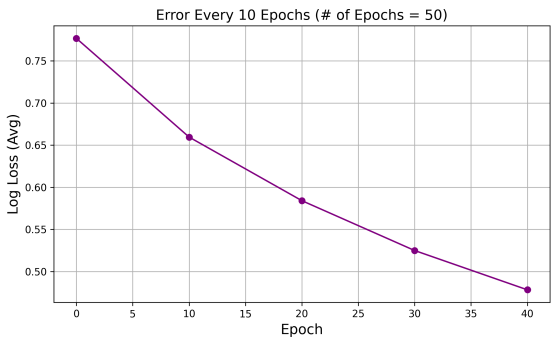
Part #2: Implementing Perceptron via Gradient Descent

Plotting data from `data.csv`, we implement perceptron using the *Gradient Descent* approach. Here, we plot the initial separation line as `red`, subsequent ones after each iteration in `dashed green`, and the last one in `black`. Here we trained and ran with differing epoch values of 50, 200, and 1000 and differing learning rates of values 0.001, 0.1 and 0.5. Additionally, we computed log loss (error) and plotted the error graph every 10 epoch.

Starting with $r = 0.01$ and $epoch = 50$, we can see that the initial red separation line slowly rotating through the fan of the dashed green lines towards the black line, which is located at a reasonable spot between the blue circles and orange squares.

Unlike the Heuristic approach, we can see that the green lines converge much tighter and closer to each other rather than fluctuating wildly, showing the smoother gradient based updates.





Additionally, the error graph at $epoch = 50$, confirms that the gradient descent is working as it should. With the average epoch loss at $epoch = 0$ being 78 and dropping to about 0.48 $epoch = 40$, showing a smooth decreasing curve. However, the loss is still declining and going down at the end of training and hasn't fully flattened out, showing that the model has not fully converged.

With a small epoch value and learning rate, we can see that the updates are moving in the right direction but too slow to reach a minimum. So from here, we can assume that increasing the epoch value and/or learning rate would result in the loss to continue decreasing and the boundary line to settle into a better and more optimal position.

Including snippets of the code from `perceptron_gd.py`. The follow is the sigmoid activation function, initialization of random weights and bias, actual training loop with gradient descent updates, and the log loss computation.

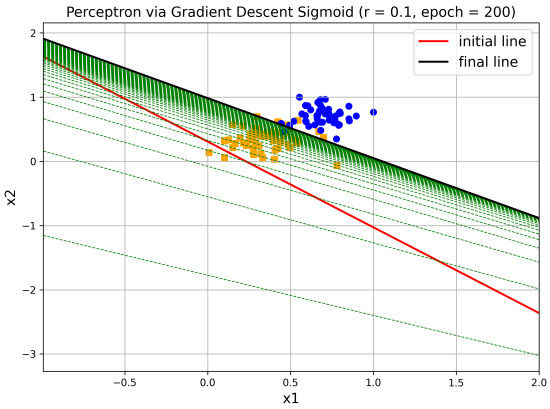
```python
# Sigmoid Function
def sigmoid(z):
    if z < -500:
        z = -500
    if z > 500:
        z = 500
    return 1.0 / (1.0 + math.exp(-z))

# Step #1: starting with random weights and bias
random.seed(0)
w1  = random.uniform(-1, 1)
w2  = random.uniform(-1, 1)
b   = random.uniform(-1, 1)

r = 0.5
num_epochs = 1000
```

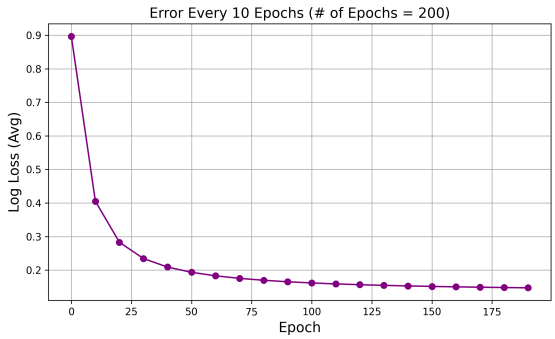
```python
# Step #2 & Step #3: training using Gradient Descent
errors_per_epoch = []

for epoch in range(num_epochs):

    for i in range(n_points):
        x1  = x1_list[i]
        x2  = x2_list[i]
        y   = labels[i]

        z = w1 * x1 + w2 * x2 + b
        y_hat = sigmoid(z)

        err = y - y_hat

        b   = b + r * err
        w1  = w1 + r * err * x1
        w2  = w2 + r * err * x2

    # Calculate the total log loss across all of the points
    total_loss = 0.0
    for i in range(n_points):
        z = w1 * x1_list[i] + w2 * x2_list[i] + b
        y_hat = sigmoid(z)

        if y_hat < 1e-12:
            y_hat = 1e-12
        if y_hat > 1 - 1e-12:
            y_hat = 1 - 1e-12
        y = labels[i]
        total_loss = total_loss - (y * math.log(y_hat) + (1 - y) * math.log(1 - y_hat))
    avg_loss = total_loss / n_points
    errors_per_epoch.append(avg_loss)
```

With the code snippets above, the gradient descent approach starts with some random weights and bias, but instead of a set and hard 0 and 1 step function, it uses sigmoid in order to produce a continuous prediction between 0 and 1. And for every point, it calculates and computes the error and scales the weight update by that calculated error. And after each epoch, the log loss is also computed across all of the points in order to track how well the model is performing, with a production of a smooth downward error curve as the boundary converges toward the best and optimal spot.

Next, with $r = 0.1$ and $epoch = 200$, we can see that the separation plot showing the red initial line rotating through the clustered and tight green dashed lines towards the final black line, which clearly and successfully separates the blue circles and orange squares.

Unlike the previous run, the boundary converges much faster and a lot further and resulting in the final black line being positioned much better. The clumped up and tight collection of the green dashed lines confirms a stable and non oscillating convergence.

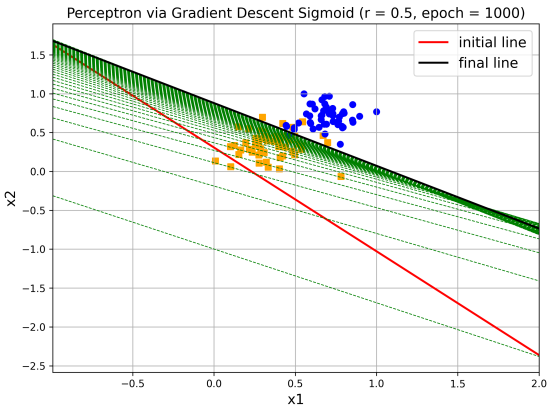


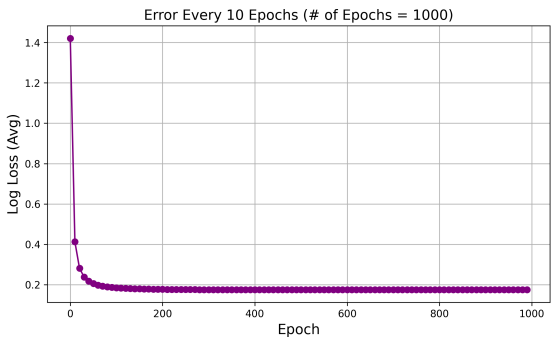


Additionally, the error graph shows the average loss dropping from 0.90 at epoch 0 to about 0.20 at epoch 40, then gradually flattening out at about 0.15 as training goes to epoch 200. This smooth line and flattening shows the gradient descent reaching a minimum. So unlike the previous run with $r = 0.01$, the curve at this run has plateaued and flattened out, showing and indicating that the model has effectively converged.

And now with $r = 0.5$ and $epoch = 1000$, we can see that its very similar to the previous runs in regards to its red initial lines and black separation line. But here, with a larger learning rate, the boundary goes back into its final position a lot quicker.

At this run, below, it shows that the model settles early on and made small changes and refinements for the remaining epochs. Additionally, looking at the error graph, it confirms rapid convergence. Here the average loss drops from about 1.42 at epoch 0 to about 0.20 within the first 50 epochs, and then flattens out to around 0.17 where it stays flattened and constant for the rest of the 950 epochs. This result shows that the larger learning rate along with the bigger epoch amount, effectively reaches the minimum achievable log loss for this dataset.





Looking at the output below, the final weights being:

$$w_1 = -16.455872670713813$$
$$w_2 = -20.394294553344334$$
$$b = 17.93152692258983$$

And final $log_{loss} = 0.17513370515531024$, these data confirms that the gradient descent perceptron was able to converge successfully. The large weight indicates that the sigmoid is making confident, near 0 or near 1 predictions after 1000 epochs of training. The residual loss amount of 0.17 shows the effective minimum for this dataset, since the two classes and data types overlapping slightly, a perfect separation is not possible.

```
(.venv) jomcgoat@Josephs-MacBook-Pro Assignment_6 % python perceptron_gd.py
Dataset Shape: (100, 3)
First 5 Rows:
        x1        x2  label
0  0.78051 -0.063669      1
1  0.28774  0.291390      1
2  0.40714  0.178780      1
3  0.29230  0.421700      1
4  0.50922  0.352560      1

No missing values found.

Final Weights: w1 =  -16.455872670713813  w2 =  -20.394294553344334  b =  17.93152692258983
Final Log Loss:  0.17513370515531024
```